

# Information Retrieval and Question Answering

Jonathan May

October 20, 2025

## 1 Information Retrieval

You can take entire courses on information retrieval (e.g., CSCI 572, Stanford’s CS 276), and it’s not always considered the same thing as NLP, but it’s a general problem which question answering (QA) is an instance of, and QA is squarely an NLP task.

IR is probably the NLP application you use most often and the most commercially successful NLP application – search (like web search) is an IR problem. The general idea is that you have a (very large) collection of **documents** that for our purposes, you can think of as text strings, usually paragraphs or more long. You then have a **query**, which is a short phrase, natural language of some sort, that indicates the kind of document you’re looking for. In general, the task of IR is to **rank** all the documents in order based on how much they satisfy the goals of the query. In practice, all but a small number of documents are ranked at  $\infty$  and the rest are chosen, so you can think of that as a way of **classifying** each document as matching or not.

### 1.1 IR evaluation methods

Generally, we have a set of held-out queries and documents, along with annotations of documents that are relevant for each query. The document set is typically very large and the relevant document set for a query is much smaller than the background data. It also is generally not a perfect recall since this would require checking every document in a collection (background assumed not to match).

#### 1.1.1 Precision and Recall @K

Standard precision and recall seem valid, but as there are multiple relevant documents per query, we must consider how many documents we are allowed to return.

Precision@K = You got N queries and for each query, K docs. Now, if you have a total of H relevant documents, then  $P@K = \frac{H}{NK}$ .

Recall@K = You got N queries and for each query, K docs. Out of these NK docs, there are H relevant documents, then  $R@K = \frac{H}{\text{All relevant documents}}$ . If overall more than K relevant docs exist you can’t get 100%.

None of these take the position in the K you return into account.

### 1.1.2 Mean Average Precision (MAP)

Consider a ranked list of documents per query. Take the average of precision over several K. Example for K of 5, with 3 hits:

- $1/1/1/0/0 = \frac{1}{5}(1 + 1 + 1 + \frac{3}{4} + \frac{3}{5}) = .87$
- $0/0/1/1/1 = \frac{1}{5}(0 + 0 + \frac{1}{3} + \frac{2}{4} + \frac{3}{5}) = .2867$

That's the Average Precision. Take a mean across all queries, and you get MAP = Mean Average Precision.

Other metrics that you can look up:

- nDCG (Normalized Discounted Cumulative Gain) – Take how relevant a document is into account and penalize for being lower on the list
- MRR (Mean Reciprocal Rank) – For a query, if the first relevant hit is at position  $r$  in your list, score  $\frac{1}{r}$ . Average across all queries.
- User results – From clicks, eye-tracking, rarely from reported relevance results.

## 1.2 Vocabulary Models

I'm sure you're thinking, 'ok, this is going to be old stuff and then we'll see the neural stuff that works much better.' In a way, this is true, but if you're searching for something very specific, just making sure the words you search for are matched is better than the more 'semantic' neural methods and so (we think) some form of these exact match approaches are still employed in commercial search.

### 1.2.1 Not Covered

- SQL
- Other logical forms (how to exclude something)

### 1.2.2 Inverted Index Lookup

The straightforward approach is 'if the words of the query are in the document, return the document.' This is often what I want from, say email. Like, I know there was an email about when this class was held in 2020, So I would search `csci 662 fall 2020 schedule` and hopefully, I'd find the email that has those words in it.

Complications:

- What if the email says `cs662` (tokenization)?
- What if the email says `autumn` (synonym)?
- What if the email says `schedules` (normalization)?

- What if I have 25 million emails to look up? (scalability)?

To solve the first three, you do some standard document preprocessing. To do the last, you use an inverted index. For each term in the vocabulary, you save a list of documents in sorted order that have that word. Then, you can easily do an intersection.

(example from Chris Manning's slides)

$t1 : 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 32 \rightarrow 64 \rightarrow 128$

$t2 : 1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 8 \rightarrow 13 \rightarrow 21 \rightarrow 34$

---

**Algorithm 1** Intersect

---

**Require:** sorted queues/min heaps  $v_1, v_2$

**Ensure:**  $a$  contains the elements common to both  $v_1$  and  $v_2$

```

 $a \leftarrow \{\}$ 
while  $v_1 \neq ()$  and  $v_2 \neq ()$  do
  if  $v_1.\text{head} = v_2.\text{head}$  then
     $a \leftarrow a \cup \{v_1.\text{head}\}$ 
     $v_1.\text{pop}()$ 
     $v_2.\text{pop}()$ 
  else if  $v_1.\text{head} < v_2.\text{head}$  then
     $v_1.\text{pop}()$ 
  else
     $v_2.\text{pop}()$ 
  end if
end while

```

---

### 1.2.3 TF-IDF

If all your query words match the document, then you have a match, but if only some of them match, then what? Should it just be most of them that match? No, all words are not equally relevant. Intuitively, content words matter more. So, how do we quantify this?

**Term Frequency** is an important characteristic. If a word you search for occurs 10 times in document 1 and 1 time in document 2, it's probably more important in document 1. But is it 10 times more important? Probably not, so the rule of thumb is, for term (word)  $t$  occurring  $f$  times in document  $d$ , the term frequency rate  $tf_{t,d}$  is:

$$tf_{t,d} = \begin{cases} \log_{10}(1 + f), & \text{if } f > 0 \\ 0, & \text{otherwise} \end{cases}$$

Why add 1? Because the log of 1 is 0, which would be awkward. Why set it to 0 if  $t$  doesn't exist? Because the log of 0 is undefined, which is even more awkward. Practicalities!

**Term Scarcity** is also important! What a contradiction! But it's true – if your query is 'the yankees', it's much more important to match 'yankees' than 'the.' Let  $N$  be the number of documents in your collection. Let  $d$  be the number of documents the term  $t$  appears in. Then the inverse document frequency rate  $idf_t$  is:

$$idf_t = \log_{10}\left(\frac{N}{d}\right)$$

This only really matters for queries with more than one term, of course. But we weight a term  $t$  by  $tf.idf_{t,d} = tf_{t,d} \times idf_t$  and can represent the score for a query-document pair as the sum of the  $tf.idf_{t,d}$  for all  $t$  in both query and document.

#### 1.2.4 BM25

BM25 is a fancier version of TF-IDF that doesn't punish documents for being short. SotA for lexical retrieval and better than you might think.

$$BM25_{t,d} = idf_t \cdot \frac{f \cdot (k + 1)}{f + k \cdot (1 - b + b \cdot \frac{|d|}{\text{average length of all documents}})}$$

$k$  and  $b$  are hyperparameters, typical values for them are:  $k = 2$  and  $b = 0.75$ .

### 1.3 Dense Vector Models

Of course, these methods only work if you have an exact match of some terms. We tried to avoid mismatches with tokenization, normalization, and stemming, but we still might not guess the exact words. So, you guessed it, we can try using sequence representations, where it's assumed that two sequences that are similar will have a small cosine distance (note we are relying on the semantics to be pretty much only synonymy.) This is sometimes lumped under the term "Dense Passage Retrieval"

#### 1.3.1 Transformer (cross-encoder) Approach

Since we do have lots (hundreds of thousands of examples) of training data, we could just use BERT, right? We would encode the query, a [SEP] token, then the document text, and learn a binary classifier to predict whether the document matches the query.

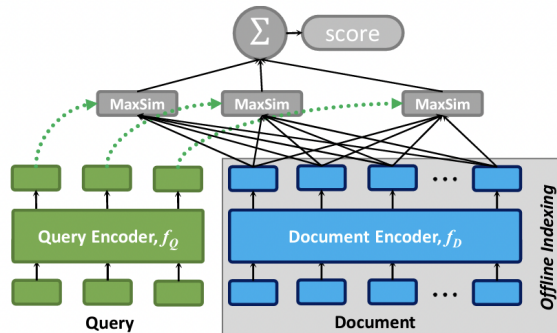
Would this work? Yes and no. This approach will give good performance on lots of IR corpora, but in order to use it you have to pair each query with each document. At web scale that's impossible.

#### 1.3.2 Bi-encoder Approach

A lightweight approach uses two (hence bi-) encoders, one to encode all the documents and one to encode the query. At training time, you optimize to maximize dot product/minimize cosine distance of the query and its matching documents; you can have two separate encoders (perhaps BERT-based), one that gets optimized jointly, and lots of other variants are possible, too. At inference time, you take your entire corpus and encode it, then for each query you get the nearest neighbors. There are some really fast approaches for this; I like the Faiss library.

### 1.3.3 Late-stage Interaction

A popular approach is ColBERT, which does not consider queries or documents to be single-sized vectors but rather considers each contextual token representation individually. To determine the similarity between a query and a document, for each token in the query, the contextual token in the document that is closest is identified. The maximum similarity for each query token is then summed. Doing this for every document would be infeasible, so typically in a first pass, for every query token, the maximum matching contextual token for any document is obtained, then, in a second pass, only the documents corresponding to initial maximum matches are returned.



## 1.4 Training

It's not sufficient to hope that a query and matching document will be together in the semantic space. They may not be actually semantic similar so we have to train them to exhibit this behavior. This is generally framed as a multi-class classifier problem: given a query, which of  $n$  documents is preferred?

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(\mathbf{q}_i, \mathbf{p}_i)/\tau)}{\exp(\text{sim}(\mathbf{q}_i, \mathbf{p}_i)/\tau) + \sum_{j=1}^m \exp(\text{sim}(\mathbf{q}_i, \mathbf{n}_j)/\tau)}$$

for queries  $q$ , positive docs  $p$ , negative docs  $n$ . similarity is typically cosine but other similarities have been tried.

This can also be viewed as contrastive learning. But to do this we need examples. Typically datasets such as MSMARCO have queries, large document 'haystacks', and one or more selected docs from the haystacks that are 'needles' or positive results. But we can't have all other docs to be negative; its too many. so which to choose?

If we assume all non-positive docs are negative, an easy and efficient approach is 'in-batch' sampling. Per training batch, all the rows that are not your row are considered negative. So you can efficiently access them for negative choices with a little creativity.

But most negatives are very obviously negative. It's more effective to find so-called 'hard negatives' for each query to make for a more effective classifier. Where to get them from? You can use a weaker IR engine such as BM 25 or in-batch-trained DPR. After building the weaker system, predict documents for your training queries. Documents it returns that are not labeled as positive can be taken as hard negatives. Of course, a concern is that in actuality, not all positive associations are ever recorded in any training data set. Thus, the retrieved "hard negatives" may, in fact, be simply unlabeled positives. If this happens too

often, you can actually make your new system worse because it will be too confused, having trained on mislabeled examples.

## 2 Question Answering

Question Answering (QA) is a special kind of IR where the query is posed in the form of a question, and rather than a document, we typically want a fact (that often comes from a document).

### 2.1 Open Domain QA

If there's a big document collection you can treat the problem like IR, at least at first. Simply(?) rephrase the question as a query (e.g. 'What is the capital of New York' → 'New York capital'), then retrieve matching documents, then proceed as if you've been given the passage, as described below.

Or we can make things a little easier by just providing the document. That is often what is done; see the list of datasets (section 2.5) below.

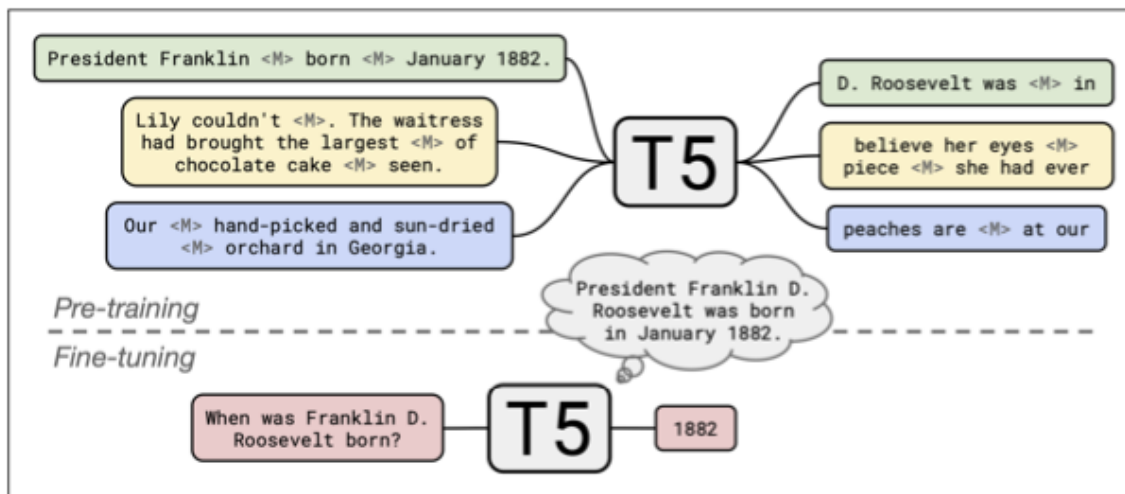
### 2.2 Extractive Factoid QA

Now that you have a document, we need to whittle it down to find which sentence contains the answer. Here, a single BERT model might actually work – encode the question and each sentence, and binary classify. The scale is not so big.

Finally, we need to find the actual passage. An IE or other span-based model simply needs to find the beginning and the end of the selected sentence. Assuming nothing has gone wrong, you now have your answer!

### 2.3 Abstractive QA

What if the exact answer isn't in the text but a reasonable person could figure it out? What if we just want to train a big model with a lot of text and ask it questions? GPT-3 can do that, as can T5, which is an encoder-decoder model trained using the paradigm in the figure below.



## 2.4 Classic Approach

QA is really old and the classic approach is a pipeline of multiple systems. Even now those pipelines can often outperform slick neural models. Here's a brief list of the steps:

- Figure out the answer type (Is 'Which US state capital has the largest population?' asking for a number, a state, or a city?)
- Figure out the focus (what words in the question should be replaced by the answer?)
- Based on the answer type, extract entities of the correct type from the retrieved documents
- Rerank all the candidate answers

Something like this was used by Watson to beat experts like Ken Jennings in 2011. There is current work on doing well in "quiz bowl" environments where context information and the question itself are progressively revealed and answering early yields more points.

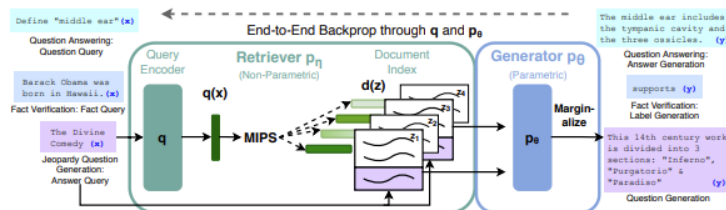
## 2.5 Datasets

- SQuAD (Stanford Question Answering Dataset, 2016) – Annotators took passages from Wikipedia, then made up questions whose answers were in the passages. SQuAD 2.0 (2018) also includes unanswerable questions – 150k total questions. If you include the passages it's extractive QA, if you throw them away it's open-domain with wikipedia as the document corpus.
- HotpotQA (2018) – Take several documents and make a question that you need to read and reason from all of them to make a decision. Requires multi-hop reasoning.
- TriviaQA, Natural Questions (2017, 2019) – Were written by trivia people/google users, and also include contexts with the answer, but weren't written by unconsciously biased annotators.

- TyDi QA (typologically diverse) (2020) – 204k QA pairs from 11 different languages.

### 3 Retrieval Augmented Generation (RAG)

Although largely based on a paper from 2021,<sup>1</sup> in 2023 (and beyond) RAG is very popular due to the hallucination crisis observed in ChatGPT. The key idea here is to combine both retrieval and generation so that generated information is based on actual retrieved documents that have some basis in reality. Intuitively, this makes sense. If you are talking with someone and they ask about something fact-based that you don't know the answer to off the top of your head, you might go look it up (in an encyclopedia, online, etc.). Having read the document, you can then express the right information based on the context of the conversation. That's exactly what's done as depicted below:



The idea is pretty simple: you want to generate  $y$  from  $x$ , but first, you ask an IR model for some documents  $z$ . Then you generate  $y$  in the context of both  $x$  and  $z$  (literally concatenate them). Generally, RAG marginalizes over several (top- $k$ ) documents  $Z$ , so the likelihood is as follows:

$$p(y|x) \approx \sum_{z \in Z} p(z|x)p(y|x, z)$$

The query and document encoders are BERT models and the generator is a BART model. In the original paper, the query encoder and the generator are fine-tuned, but the document encoder isn't since that would require re-encoding the document corpus (Wikipedia) at each step. In current RAG systems, nothing is fine-tuned since the models are all too big, so essentially RAG is just a paradigm for generation – include retrieved documents when generating (and then possibly do a more complicated joint decoding or rerank candidate outputs). RAG is actively being investigated in the context of super good generation models so many innovations are sure to emerge beyond the scope of what is taught here.

It's big business too. In 2025<sup>2</sup> some RAG-based startups are OpenEvidence (raised \$210m on 3.5b value), which provides medical search for clinicians<sup>3</sup>, Abridge (\$300m on 5.3b), which documents hospital workflows,<sup>4</sup> and Glean (\$150m on 7.3b), which deploys RAG-enabled search/agent for any company. Plus lots more.

<sup>1</sup><https://arxiv.org/pdf/2005.11401v4.pdf>

<sup>2</sup><https://tinyurl.com/26hzq26c>

<sup>3</sup><https://pmc.ncbi.nlm.nih.gov/articles/PMC12159471/>

<sup>4</sup><https://themelan.com/abridge-ai-medical-notes-charting-epic/>